

Docker challenges

- >Create a customized Docker image using a Dockerfile.
 - >Push the image to Docker Hub.
 - >Push the same image to Amazon ECR.
 - >Provision one EC2 instance using Terraform and install Jenkins.
 - >Create one Jenkins job to build and push the Docker image to Docker Hub.
1. Source: <https://github.com/betawins/Python-app.git>



Source Codes: <https://github.com/betawins/docker-tasks.git>

From the **frontend source code**, write a Dockerfile, build a Docker image, run it, and push that image to your Docker registry.

From the **Java-based source code**, write a Dockerfile, build, run, and push the image to the Docker registry.

From the **Node.js-based source code**, write a Dockerfile, build with tag **v1**, run, and push it to the Docker registry.

- Write a **docker-compose file** to set up WordPress with a MySQL database.

1.Create a customized Docker image using a Dockerfile.

Step 1: Project Structure

```
mkdir nginx-app
cd nginx-app
Create a simple web page:
```

```
vi index.html
```

```
<h1>Hello from NGINX Docker Image</h1>
```

```
[root@ip-172-31-20-146 my-app]# vi dockerfile
[root@ip-172-31-20-146 my-app]# vi index.html
[root@ip-172-31-20-146 my-app]# docker build -t waseem/nginx-app .
[+] Building 3.5s (8/8) FINISHED
```

Step 2: Write Dockerfile

```
vi Dockerfile
```

```
FROM nginx:alpine
COPY index.html /usr/share/nginx/html/index.html
EXPOSE 80
```

```
[root@ip-172-31-20-146 my-app]# cat dockerfile
FROM nginx:alpine
COPY index.html /usr/share/nginx/html/index.html
EXPOSE 80

[root@ip-172-31-20-146 my-app]#
```

Step 3: Build Docker Image

```
docker build -t waseem/nginx-app .
```

Step 4: Verify Image Created

```
docker images
```

```
[root@ip-172-31-20-146 my-app]# docker build -t waseem/nginx-app .
[+] Building 3.5s (8/8) FINISHED
=> [internal] load build definition from dockerfile
=> => transferring dockerfile: 175B
=> [internal] load metadata for docker.io/library/nginx:alpine
=> [auth] library/nginx:pull token for registry-1.docker.io
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [internal] load build context
=> => transferring context: 137B
=> [1/2] FROM docker.io/library/nginx:alpine@sha256:1d13701a5f9f3fb01aaa88cef2344d65b6b5bf6b7d9fa4
=> => resolve docker.io/library/nginx:alpine@sha256:1d13701a5f9f3fb01aaa88cef2344d65b6b5bf6b7d9fa4
=> => sha256:c032460d1fd73978317479ba23c37bcb57d93156cab122eb3c54b8e4bdc292fa 2.50kB / 2.50kB
=> => sha256:b76de378d57272a1dd9091a05dd548a3639dfb792ebdbf95d06704d2950afdea 11.65kB / 11.65kB
=> => sha256:6b7b6c7061b76cdb8601e18722d12ae3232f0ddcfa1d2983754abcc6ce0a8a83 954B / 954B
=> => sha256:1d13701a5f9f3fb01aaa88cef2344d65b6b5bf6b7d9fa4cf0dca557a8d7702ba 10.33kB / 10.33kB
=> => sha256:bca5d04786e112d958f100a66f8257b2aeefc14b64d81e405c3c44acff2fb000 1.86MB / 1.86MB
=> => sha256:3e2c181db1b0985ce357c7aaf48ac615f30f392cd15d5c5ba34c4faa1f4f39a2 626B / 626B
=> => sha256:399d0898a94e0084f81499a3e3c29824357118c7ce551648ea1dbab813884661 404B / 404B
=> => extracting sha256:bca5d04786e112d958f100a66f8257b2aeefc14b64d81e405c3c44acff2fb000
=> => sha256:955a8478f9aceb66cbf2f579fa3c24e1af278d1fa3ffd3043d6260e21d2f7416 1.21kB / 1.21kB
=> => extracting sha256:3e2c181db1b0985ce357c7aaf48ac615f30f392cd15d5c5ba34c4faa1f4f39a2
=> => extracting sha256:6b7b6c7061b76cdb8601e18722d12ae3232f0ddcfa1d2983754abcc6ce0a8a83
=> => sha256:6d397a54a185dd0b6638d1a3934b81daef7a140741e12697377d6279066f7ca7 1.40kB / 1.40kB
=> => extracting sha256:399d0898a94e0084f81499a3e3c29824357118c7ce551648ea1dbab813884661
=> => sha256:5e7756927bef33a266e1221356d5da8553139cb80bc5b1b3827010811d9ea268 20.24MB / 20.24MB
=> => extracting sha256:955a8478f9aceb66cbf2f579fa3c24e1af278d1fa3ffd3043d6260e21d2f7416
=> => extracting sha256:6d397a54a185dd0b6638d1a3934b81daef7a140741e12697377d6279066f7ca7
=> => extracting sha256:5e7756927bef33a266e1221356d5da8553139cb80bc5b1b3827010811d9ea268
=> [2/2] COPY index.html /usr/share/nginx/html/index.html
=> exporting to image
=> => exporting layers
=> => writing image sha256:cec9b0176eecd00f36f2351d7c08a3739d58929f79dbcb35a7bd83832864056
=> => naming to docker.io/waseem/nginx-app
[root@ip-172-31-20-146 my-app]# docker images
REPOSITORY          TAG          IMAGE ID          CREATED           SIZE
waseem/nginx-app    latest       cec9b0176eec      7 seconds ago    62.1MB
```

Step5:

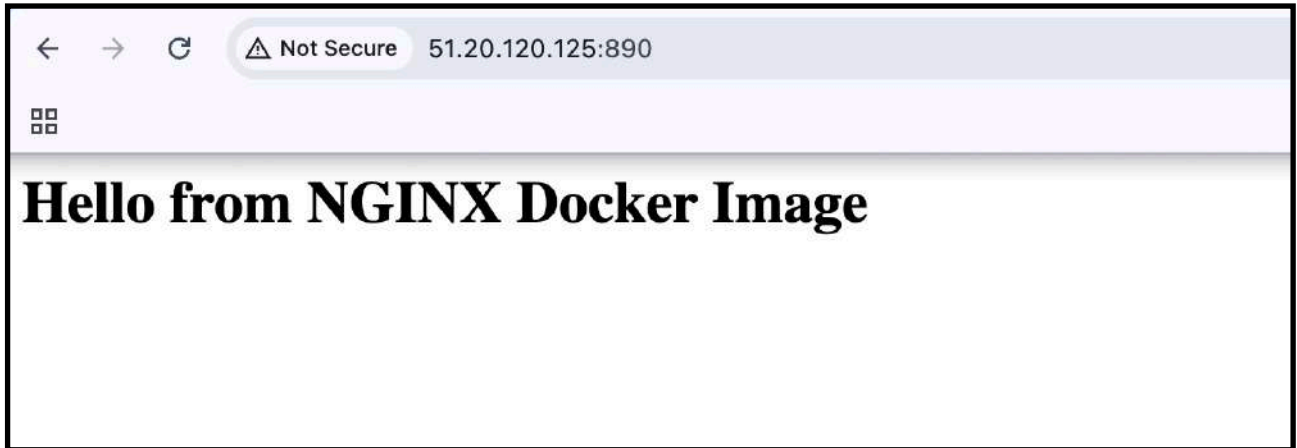
—> docker run -d -p 890:80 waseem/nginx-app .

```
[root@ip-172-31-20-146 my-app]# docker run -d -p 890:80 waseem/nginx-app
2bd80a9f8af2141c96691f9aa7e474a19a28628e95d8c42daa9a6e3b817f318a
[root@ip-172-31-20-146 my-app]# docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                               NAMES
2bd80a9f8af2   waseem/nginx-app  "/docker-entrypoint..."  11 seconds ago Up 10 seconds  0.0.0.0:890->80/tcp, :::890->80/tcp  wonderful_napi
```

—> check docker ps

Step:6

Check you ip address:890



2.Push the image to Docker Hub.

STEP 1: Login to Docker Hub

```
docker login
```

Enter:

- **Username**
- **Password**

You should see:

Login Succeeded

```
[root@ip-172-31-20-146 my-app]# docker login
Authenticating with existing credentials...
WARNING! Your password will be stored unencrypted in /root/.docker/config.json.
Configure a credential helper to remove this warning. See
https://docs.docker.com/engine/reference/commandline/login/#credentials-store

Login Succeeded
[root@ip-172-31-20-146 my-app]#
```

STEP 2: Check Your Image Name

`docker images`

Example:

```
waseem/nginx-app    latest
```

```
Login Succeeded
[root@ip-172-31-20-146 my-app]# docker images
REPOSITORY          TAG             IMAGE ID         CREATED          SIZE
waseem/nginx-app    latest          cec9b0176eec    22 minutes ago  62.1MB
jenkins/jenkins     lts             59bc435e1f03    2 weeks ago     491MB
[root@ip-172-31-20-146 my-app]#
```

Docker Hub format **must be**:

`<dockerhub-username>/<image-name>`

STEP 3: Re-tag the SAME image (no rebuild)

```
docker tag waseem/nginx-app:latest waseem08/nginx-app:latest
```

—> This does **NOT** create a new image

—> It only adds a new name (tag) to the same image

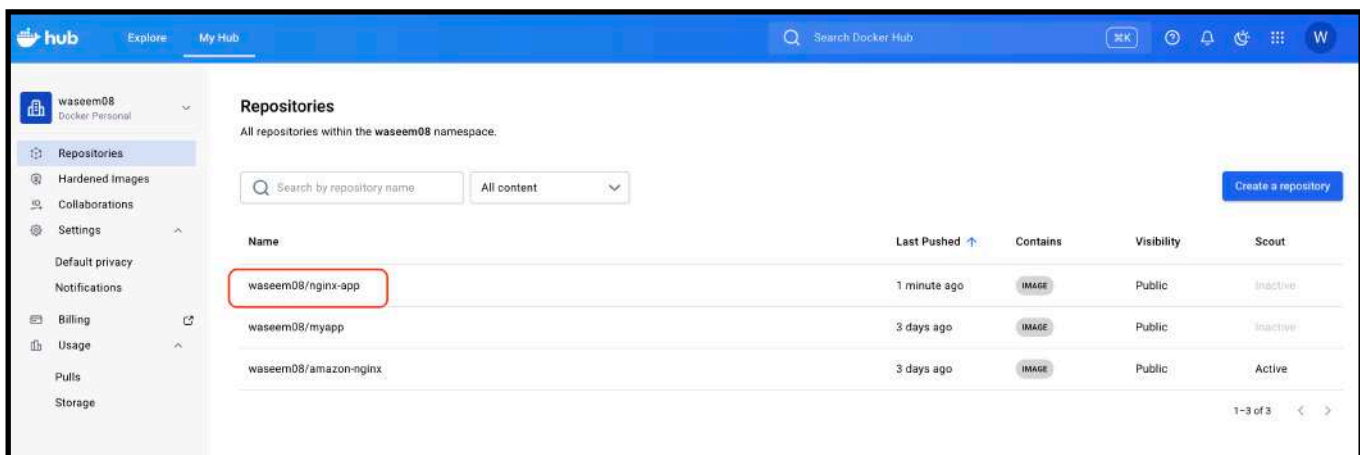
STEP 4: Push to Docker Hub

```
docker push waseem08/nginx-app:latest
```

```
[root@ip-172-31-20-146 my-app]# docker tag waseem/nginx-app:latest waseem08/nginx-app:latest
[root@ip-172-31-20-146 my-app]# docker images
REPOSITORY          TAG          IMAGE ID          CREATED           SIZE
waseem08/nginx-app   latest       cec9b0176eec     25 minutes ago   62.1MB
waseem/nginx-app     latest       cec9b0176eec     25 minutes ago   62.1MB
jenkins/jenkins      lts         59bc435e1f03     2 weeks ago      491MB
[root@ip-172-31-20-146 my-app]# docker push waseem08/nginx-app:latest
The push refers to repository [docker.io/waseem08/nginx-app]
83ecd776cacf: Pushed
da3ac26fdf0f: Mounted from library/nginx
7b2903554e63: Mounted from library/nginx
6c0e59fd138a: Mounted from library/nginx
aa0fc249df10: Mounted from library/nginx
f35bcec50d8c: Mounted from library/nginx
660f9a93104f: Mounted from library/nginx
53998a5033c3: Mounted from library/nginx
989e799e6349: Mounted from library/nginx
latest: digest: sha256:7a563c6b3f46f01eb8fd4f44381253b115d558b55af7a69d7756122c9a6b3486 size: 2196
[root@ip-172-31-20-146 my-app]#
```

Step:5 check your docker hub

—> repository



3.Push the same image to Amazon ECR.

Prerequisites:

—> Aws —configure

STEP 1: Confirm the image exists locally

docker images

You should already have:

waseem08/nginx-app latest

```
[root@ip-172-31-20-146 ~]# docker images
REPOSITORY          TAG          IMAGE ID      CREATED        SIZE
waseem08/nginx-app  latest      cec9b0176eec  37 minutes ago 62.1MB
waseem/nginx-app    latest      cec9b0176eec  37 minutes ago 62.1MB
jenkins/jenkins     lts         59bc435e1f03  2 weeks ago   491MB
[root@ip-172-31-20-146 ~]#
```

STEP 2: Create an ECR repository

(Do this once)

```
aws ecr create-repository --repository-name nginx-app --
region eu-north-1
```



```
[root@ip-172-31-20-146 ~]# aws ecr create-repository --repository-name nginx-app --region eu-north-1
{
  "repository": {
    "repositoryArn": "arn:aws:ecr:eu-north-1:207662791773:repository/nginx-app",
    "registryId": "207662791773",
    "repositoryName": "nginx-app",
    "repositoryUri": "207662791773.dkr.ecr.eu-north-1.amazonaws.com/nginx-app",
    "createdAt": "2026-02-09T09:11:08.898000+00:00",
    "imageTagMutability": "MUTABLE",
    "imageScanningConfiguration": {
      "scanOnPush": false
    },
    "encryptionConfiguration": {
      "encryptionType": "AES256"
    }
  }
}
[root@ip-172-31-20-146 ~]#
```

STEP 3: Login to Amazon ECR

```
aws ecr get-login-password --region eu-north-1 \
| docker login --username AWS --password-stdin
<ACCOUNT_ID>.dkr.ecr.eu-north-1.amazonaws.com
```

```
[root@ip-172-31-20-146 ~]# aws ecr get-login-password --region eu-north-1 \
| docker login --username AWS --password-stdin 207662791773.dkr.ecr.eu-north-1.amazonaws.com/nginx-app
WARNING! Your password will be stored unencrypted in /root/.docker/config.json.
Configure a credential helper to remove this warning. See
https://docs.docker.com/engine/reference/commandline/login/#credentials-store

Login Succeeded
[root@ip-172-31-20-146 ~]#
```

STEP 4: Tag the SAME image for ECR

```
docker tag waseem08/nginx-app:latest \
<ACCOUNT_ID>.dkr.ecr.eu-north-1.amazonaws.com/nginx-
app:latest
```



```
[root@ip-172-31-20-146 ~]# docker tag waseem08/nginx-app:latest 207662791773.dkr.ecr.eu-north-1.amazonaws.com/nginx-app:latest
[root@ip-172-31-20-146 ~]# docker images
```

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
207662791773.dkr.ecr.eu-north-1.amazonaws.com/nginx-app	latest	cec9b0176eec	46 minutes ago	62.1MB
waseem/nginx-app	latest	cec9b0176eec	46 minutes ago	62.1MB
waseem08/nginx-app	latest	cec9b0176eec	46 minutes ago	62.1MB
jenkins/jenkins	lts	59bc435e1f03	2 weeks ago	491MB

```
[root@ip-172-31-20-146 ~]#
```

—> **docker tag waseem08/nginx-app:latest
207662791773.dkr.ecr.eu-north-1.amazonaws.com/nginx-
app:latest**

—> **docker images to check the tag image**

STEP 5: Push image to Amazon ECR

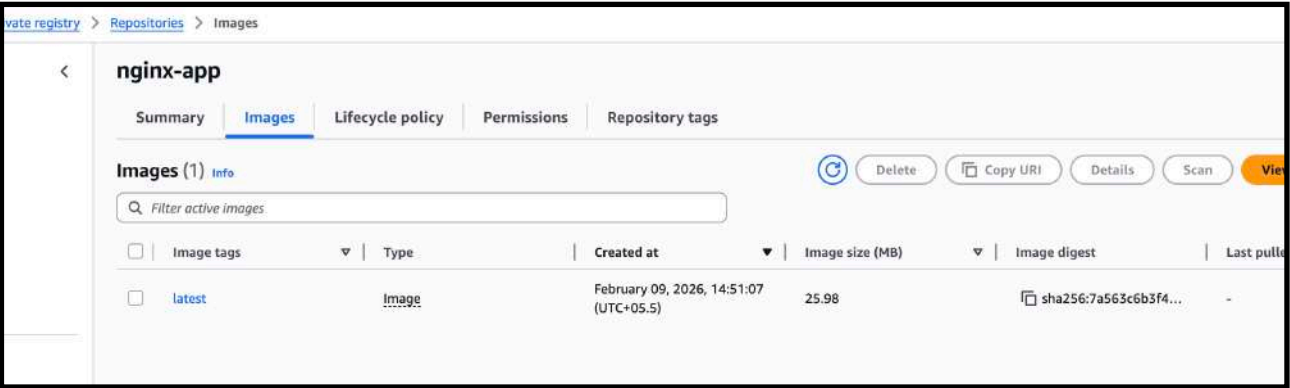
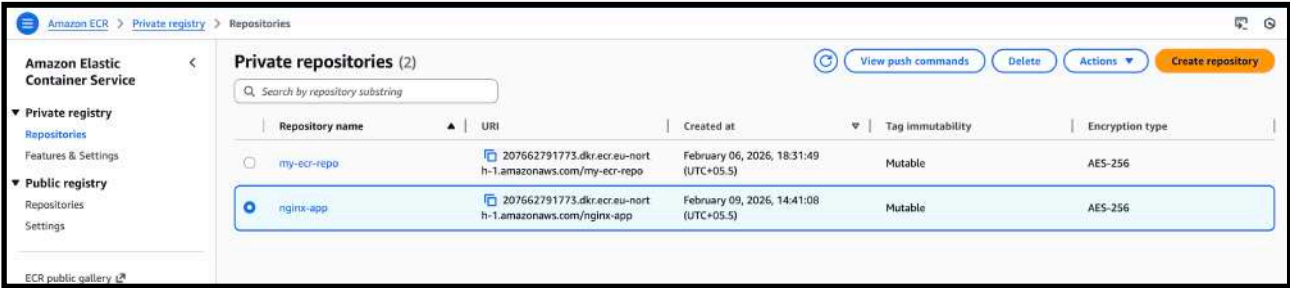
```
docker push <ACCOUNT_ID>.dkr.ecr.eu-north-1.amazonaws.com/  
nginx-app:latest
```

```
docker push 207662791773.dkr.ecr.eu-  
north-1.amazonaws.com/nginx-app:latest
```

```
[root@ip-172-31-20-146 ~]# docker push 207662791773.dkr.ecr.eu-north-1.amazonaws.com/nginx-app:latest
The push refers to repository [207662791773.dkr.ecr.eu-north-1.amazonaws.com/nginx-app]
83ecd776cacf: Pushed
da3ac26fdf0f: Pushed
7b2903554e63: Pushed
6c0e59fd138a: Pushed
aa0fc249df10: Pushed
f35bcec50d8c: Pushed
660f9a93104f: Pushed
53998a5033c3: Pushed
989e799e6349: Pushed
latest: digest: sha256:7a563c6b3f46f01eb8fd4f44381253b115d558b55af7a69d7756122c9a6b3486 size: 2196
[root@ip-172-31-20-146 ~]#
```

STEP 6: Verify in AWS Console

- Go to **Amazon ECR**
- Open **nginx-app** repository
- Image with tag `latest` will be visible



4.Provision one EC2 instance using Terraform and install Jenkins.

Prerequisites:

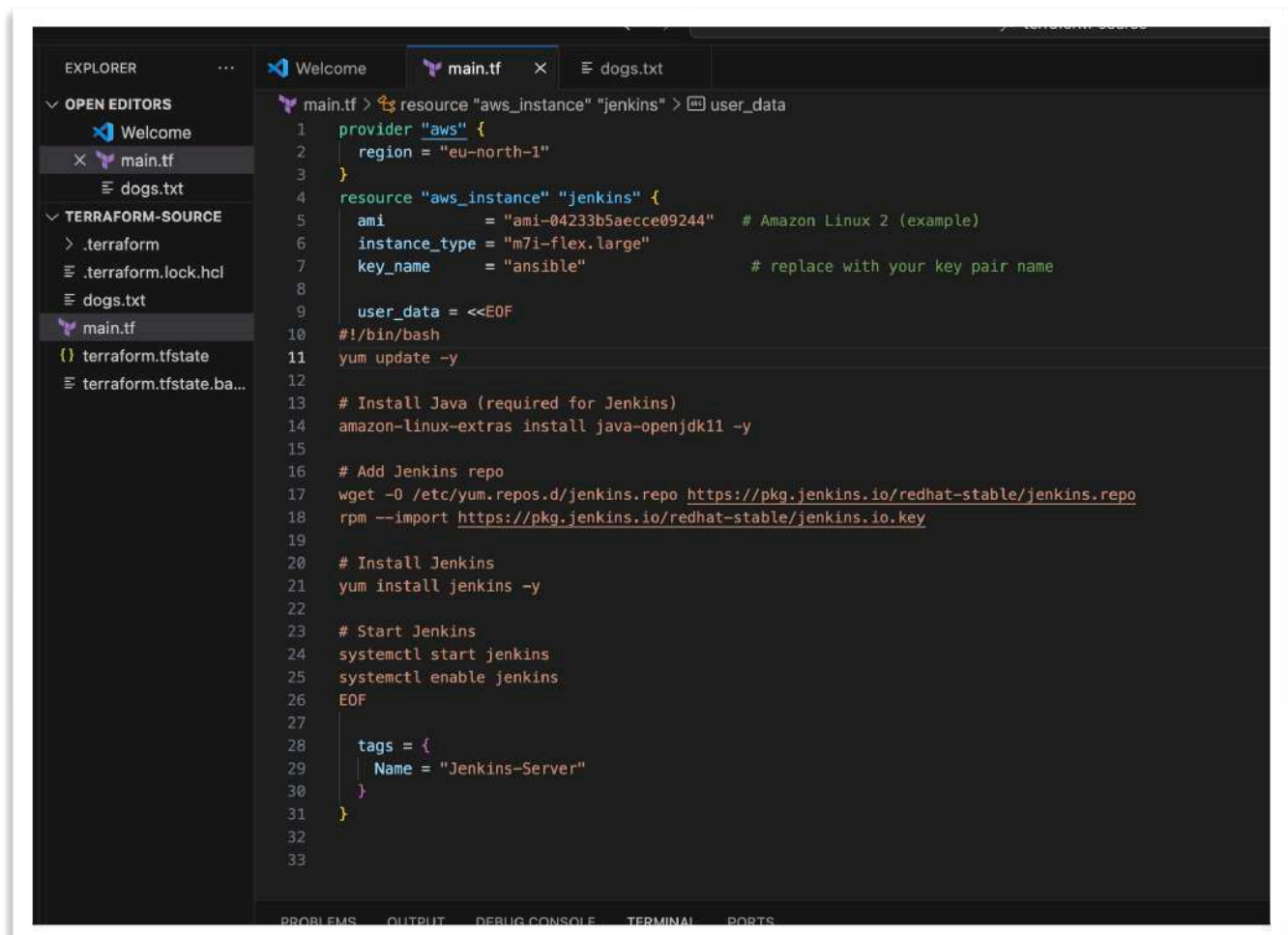
aws—configure

Steps:

- > create a directory for this task
- > mkdir terraform-source

Cd terraform-source

- > create a file name main.tf in vscode



```
1 provider "aws" {
2   region = "eu-north-1"
3 }
4 resource "aws_instance" "jenkins" {
5   ami           = "ami-04233b5aecce09244" # Amazon Linux 2 (example)
6   instance_type = "m7i-flex.large"
7   key_name      = "ansible" # replace with your key pair name
8
9   user_data = <<EOF
10  #!/bin/bash
11  yum update -y
12
13  # Install Java (required for Jenkins)
14  amazon-linux-extras install java-openjdk11 -y
15
16  # Add Jenkins repo
17  wget -O /etc/yum.repos.d/jenkins.repo https://pkg.jenkins.io/redhat-stable/jenkins.repo
18  rpm --import https://pkg.jenkins.io/redhat-stable/jenkins.io.key
19
20  # Install Jenkins
21  yum install jenkins -y
22
23  # Start Jenkins
24  systemctl start jenkins
25  systemctl enable jenkins
26  EOF
27
28  tags = {
29    Name = "Jenkins-Server"
30  }
31 }
```

—> add the above terraform code

Terraform Commands

```
+ user_data_base64          = (known after apply)
+ user_data_replace_on_change = false
+ vpc_security_group_ids    = (known after apply)

+ capacity_reservation_specification (known after apply)

+ cpu_options (known after apply)

+ ebs_block_device (known after apply)

+ enclave_options (known after apply)

+ ephemeral_block_device (known after apply)

+ instance_market_options (known after apply)

+ maintenance_options (known after apply)

+ metadata_options (known after apply)

+ network_interface (known after apply)

+ primary_network_interface (known after apply)

+ private_dns_name_options (known after apply)

+ root_block_device (known after apply)
}

an: 1 to add, 0 to change, 0 to destroy.
s_instance.jenkins: Creating...
s_instance.jenkins: Still creating... [00m10s elapsed]
s_instance.jenkins: Creation complete after 15s [id=i-08d92ff3544edbad6]

ply complete! Resources: 1 added, 0 changed, 0 destroyed.
seemakram@waseem terraform-source %
```

```
terraform init
terraform apply
```

—> now go to your aws ec2

- > login to server
- > check the status of Jenkins (active)

```

#
~\##### Amazon Linux 2023
--\#####\
--\###|
--\#/
--V~'~> https://aws.amazon.com/linux/amazon-linux-2023
---
--.-.
--/_m/'
[ec2-user@ip-172-31-26-230 ~]$ sudo -i
[root@ip-172-31-26-230 ~]# systemctl status jenkins
jenkins.service - Jenkins Continuous Integration Server
Loaded: loaded (/usr/lib/systemd/system/jenkins.service; enabled; preset: disabled)
Active: active (running) since Mon 2026-02-09 10:01:04 UTC; 6s ago
Main PID: 21807 (java)
Tasks: 53 (limit: 9307)
Memory: 750.7M
CPU: 11.544s
CGroup: /system.slice/jenkins.service
└─21807 /usr/bin/java -Djava.awt.headless=true -jar /usr/share/java/jenkins.w
Feb 09 10:01:01 ip-172-31-26-230.eu-north-1.compute.internal jenkins[21807]: [LF]> This ma
Feb 09 10:01:01 ip-172-31-26-230.eu-north-1.compute.internal jenkins[21807]: [LF]>
Feb 09 10:01:01 ip-172-31-26-230.eu-north-1.compute.internal jenkins[21807]: [LF]> *****
Feb 09 10:01:01 ip-172-31-26-230.eu-north-1.compute.internal jenkins[21807]: [LF]> *****
Feb 09 10:01:01 ip-172-31-26-230.eu-north-1.compute.internal jenkins[21807]: [LF]> *****
Feb 09 10:01:04 ip-172-31-26-230.eu-north-1.compute.internal jenkins[21807]: 2026-02-09 10
Feb 09 10:01:04 ip-172-31-26-230.eu-north-1.compute.internal jenkins[21807]: 2026-02-09 10
Feb 09 10:01:04 ip-172-31-26-230.eu-north-1.compute.internal systemd[1]: Started jenkins.s
Feb 09 10:01:05 ip-172-31-26-230.eu-north-1.compute.internal jenkins[21807]: 2026-02-09 10
Feb 09 10:01:05 ip-172-31-26-230.eu-north-1.compute.internal jenkins[21807]: 2026-02-09 10
lines 1-20/20 (END)
```

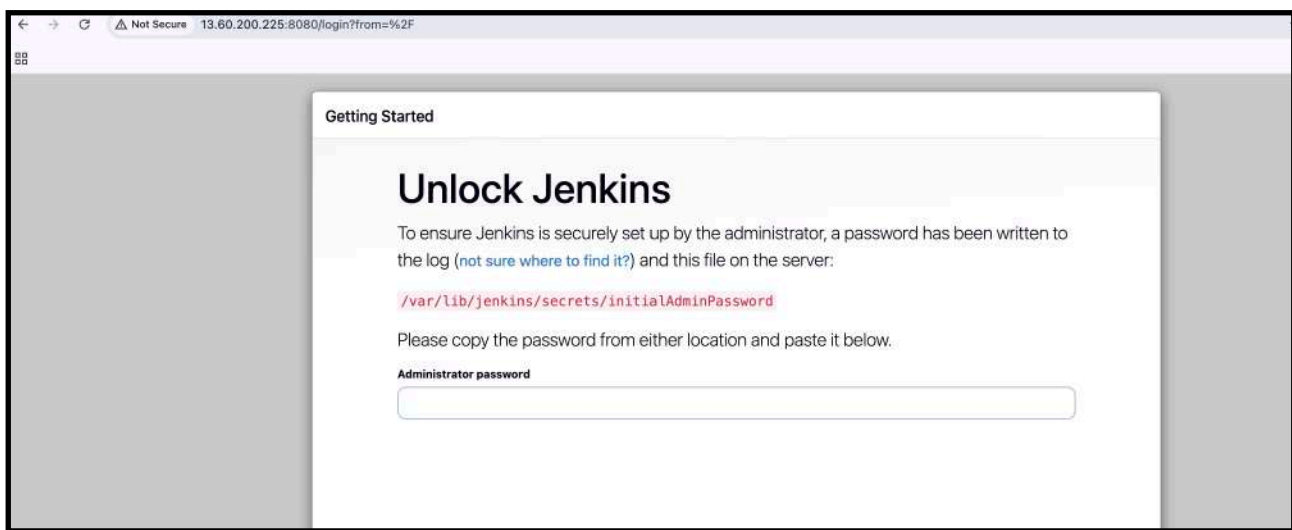
Verify the public:8080

Open:

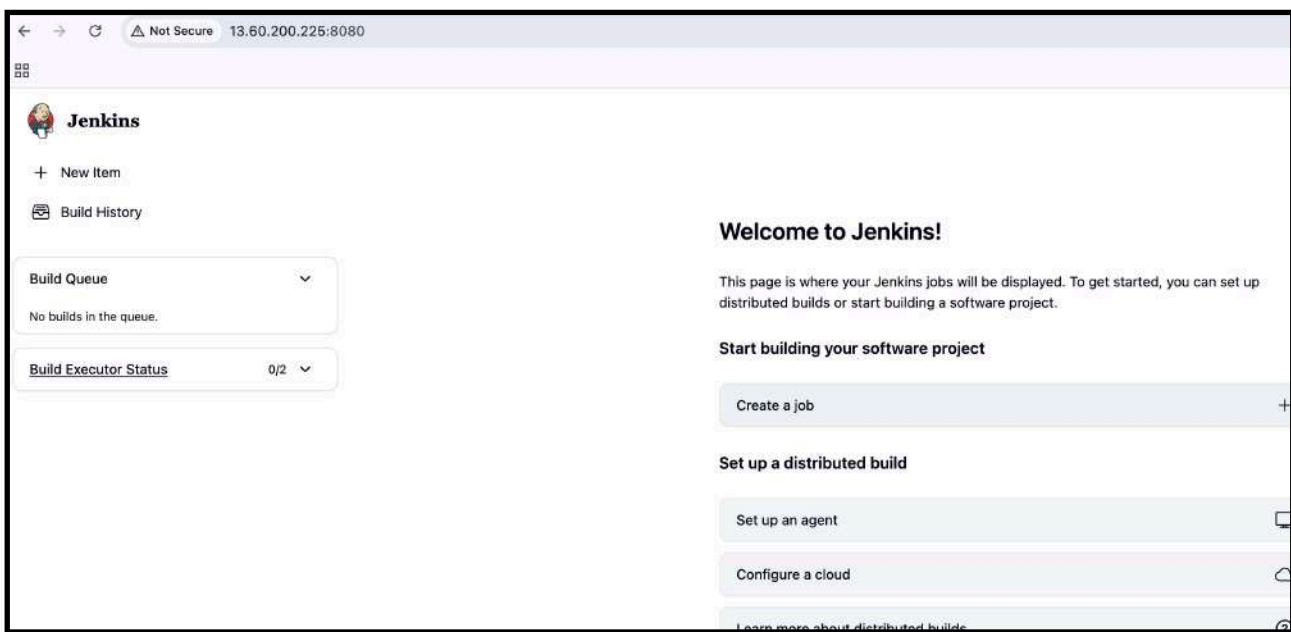
`http://<PUBLIC_IP>:8080`

Get password:

```
sudo cat /var/lib/jenkins/secrets/initialAdminPassword
```



```
[ec2-user@ip-172-31-26-230 ~]$ sudo -i  
[root@ip-172-31-26-230 ~]# sudo cat /var/lib/jenkins/secrets/initialAdminPassword  
e02d80bd722549d3af639249bd3508b0  
[root@ip-172-31-26-230 ~]#
```



5. Create one Jenkins job to build and push the Docker image to Docker Hub.

1. Source: <https://github.com/betawins/Python-app.git>

PREREQUISITES (on Jenkins EC2 – one time)

Make sure Docker works for Jenkins:

```
dnf install -y docker
systemctl start docker
systemctl enable docker
usermod -aG docker jenkins
systemctl restart jenkins
(Re-login Jenkins UI after this)
```

```
dnf install -y git (git)
```

```
[root@ip-172-31-26-230 ~]# dnf install -y docker
systemctl start docker
systemctl enable docker
usermod -aG docker jenkins
Last metadata expiration check: 0:13:53 ago on Mon Feb  9 10:00:49 2026.
Dependencies resolved.
=====
Package                                Architecture                               Ver
=====
Installing:
docker                                x86_64                                    2!
Installing dependencies:
container-selinux                     noarch                                    4
containerd                            x86_64                                    2
iptables-libs                         x86_64                                    1
iptables-nft                          x86_64                                    1
libcgroup                             x86_64                                    3
libnetfilter_conntrack                x86_64                                    1
libnftnl                              x86_64                                    1
libnftnl                              x86_64                                    1
pigz                                   x86_64                                    2
runc                                   x86_64                                    1
=====
Transaction Summary

i-0096d6953aa6a41da (Jenkins-Server)

PublicIPs: 13.60.200.225  PrivateIPs: 172.31.26.230
```

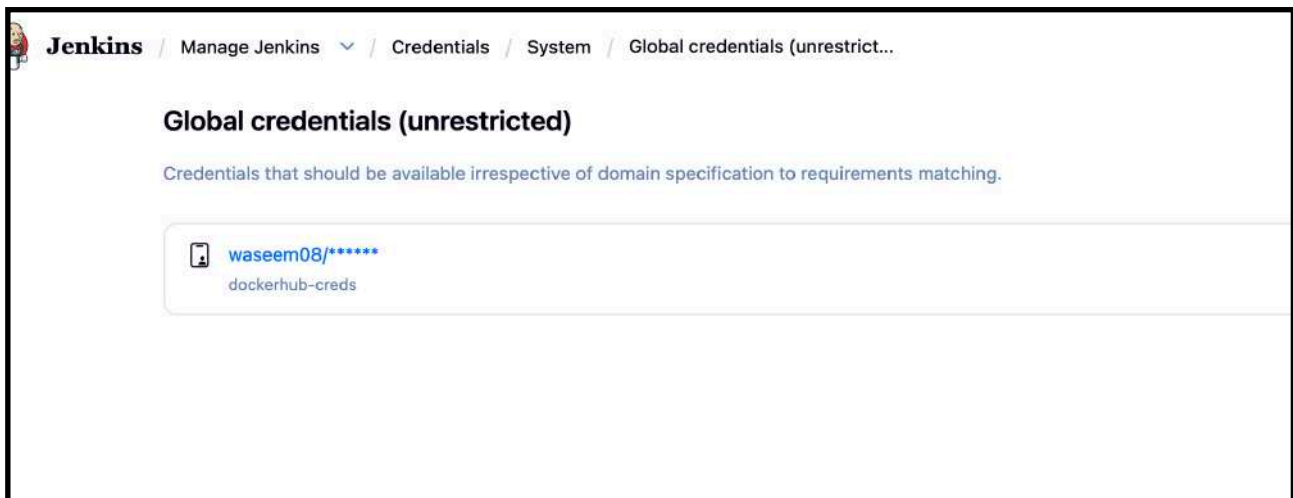
```
[root@ip-172-31-26-230 ~]# systemctl restart jenkins
[root@ip-172-31-26-230 ~]#
```


STEP 1: Add Docker Hub Credentials in Jenkins

1. Jenkins → **Manage Jenkins**
2. **Credentials** → **System** → **Global**
3. **Add Credentials**

Fill:

- Kind: **Username with password**
- Username: `waseem08`
- Password: *(Docker Hub password or token)*
- ID: `dockerhub-creds`



Step 2: Create a New Pipeline Job

1. Go to **Jenkins Dashboard** → **New Item**
2. Name it: `docker-pipeline`
3. Select **Pipeline**, then click **OK**

Step 3: Configure the Pipeline

Waseema761 / Python-app

Code Pull requests Actions Projects Wiki Security Insights Settings

main Python-app / Jenkinsfile

Waseema761 Update Jenkinsfile

Code Blame 33 lines (28 loc) · 817 Bytes

```
1 pipeline {
2   agent any
3
4   stages {
5
6     stage('Build Docker Image') {
7       steps {
8         sh 'docker build -t waseem08/python-app:latest .'
9       }
10    }
11
12    stage('Login to Docker Hub') {
13      steps {
14        withCredentials([usernamePassword(
15          credentialsId: 'dockerhub-creds',
16          usernameVariable: 'DOCKER_USER',
17          passwordVariable: 'DOCKER_PASS'
18        )]) {
19          sh '''
20            echo $DOCKER_PASS | docker login -u $DOCKER_USER --password-stdin
21          '''
22        }
23      }
24    }
25
26    stage('Push Image to Docker Hub') {
27      steps {
28        sh 'docker push waseem08/python-app:latest'
29      }
30    }
31  }
32 }
```

Add this GitHub repo url to pipeline scm

Repository URL ?

Credentials ?

+ Add

Advanced ▾

+ Add Repository

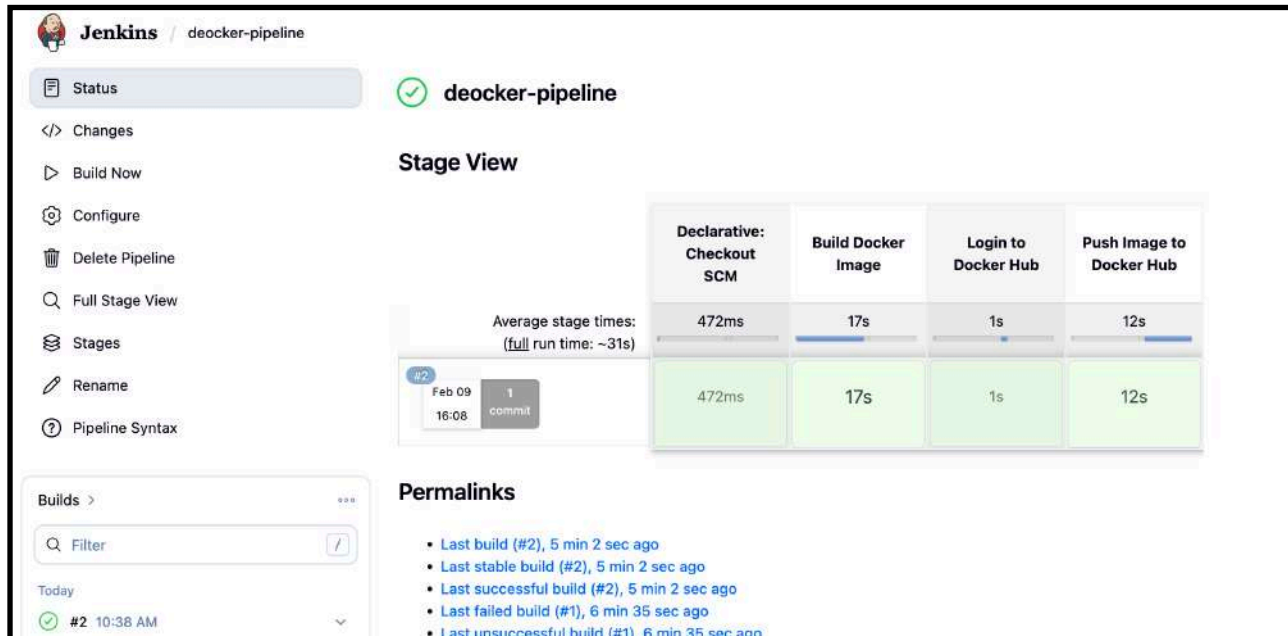
Branches to build ?

Branch Specifier (blank for 'any') ?

+ Add Branch

Step:3

—>run the job



The Jenkins Pipeline Overview for 'deocker-pipeline' shows a successful build status. The left sidebar contains navigation links: Status, Changes, Build Now, Configure, Delete Pipeline, Full Stage View, Stages, Rename, and Pipeline Syntax. The main area displays the 'Stage View' with a table of stage times and a 'Permalinks' section.

Stage	Declarative: Checkout SCM	Build Docker Image	Login to Docker Hub	Push Image to Docker Hub
Average stage times: (full run time: ~31s)	472ms	17s	1s	12s
#2 Feb 09 16:08 1 commit	472ms	17s	1s	12s

Permalinks

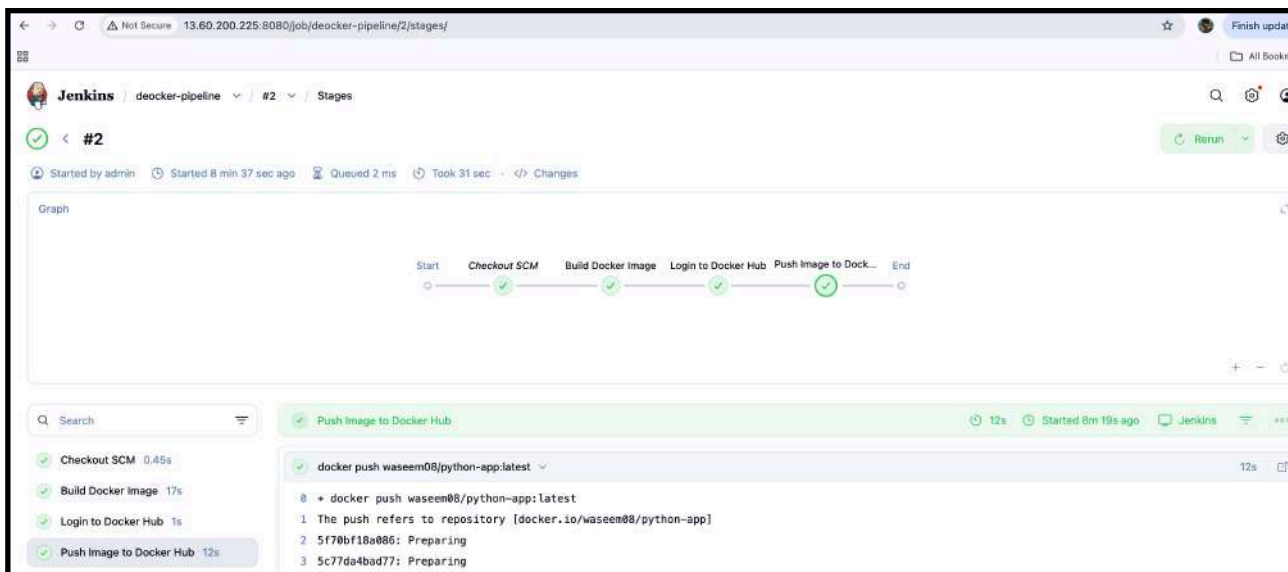
- Last build (#2), 5 min 2 sec ago
- Last stable build (#2), 5 min 2 sec ago
- Last successful build (#2), 5 min 2 sec ago
- Last failed build (#1), 6 min 35 sec ago
- Last unsuccessful build (#1), 6 min 35 sec ago

Builds

Filter

Today

#2 10:38 AM



The Jenkins Pipeline Stages View for 'deocker-pipeline' shows the build progress. The top bar indicates the build is successful and provides a 'Rerun' button. The 'Graph' section shows the build stages: Start, Checkout SCM, Build Docker Image, Login to Docker Hub, Push Image to Docker Hub, and End. The 'Push Image to Docker Hub' stage is highlighted, showing the command 'docker push waseem08/python-app:latest' and the output 'The push refers to repository [docker.io/waseem08/python-app]'.

Start Checkout SCM Build Docker Image Login to Docker Hub Push Image to Docker Hub End

Search

Push Image to Docker Hub 12s Started 8m 19s ago Jenkins

Checkout SCM 0.45s

Build Docker Image 17s

Login to Docker Hub 1s

Push Image to Docker Hub 12s

```
docker push waseem08/python-app:latest
```

1 The push refers to repository [docker.io/waseem08/python-app]

2 5f70bf18a086: Preparing

3 5c77da4bad77: Preparing

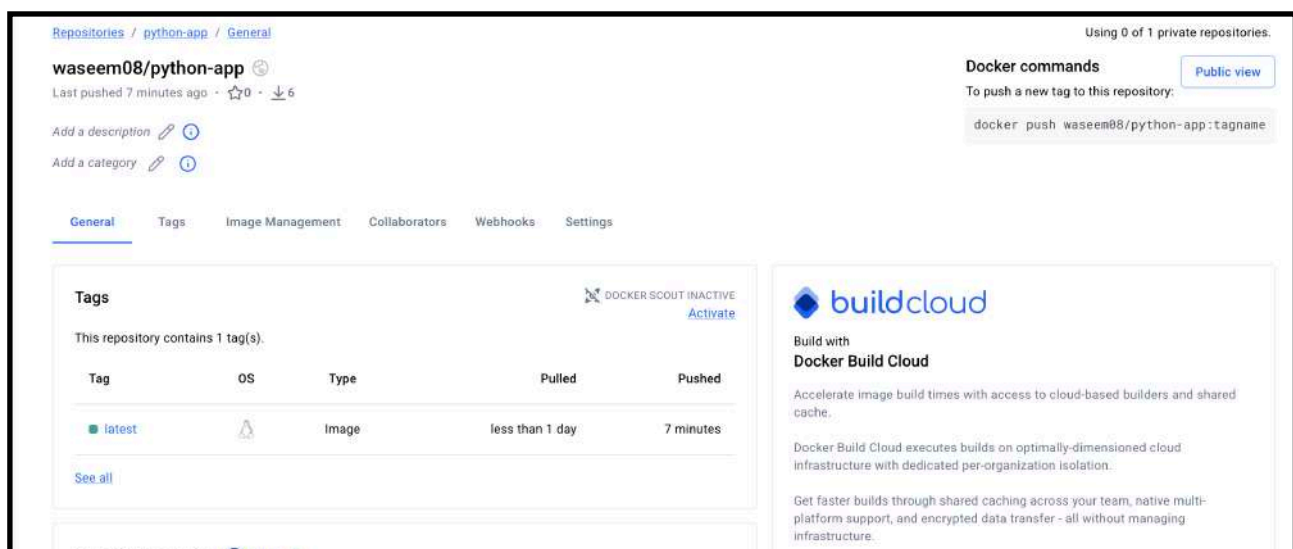
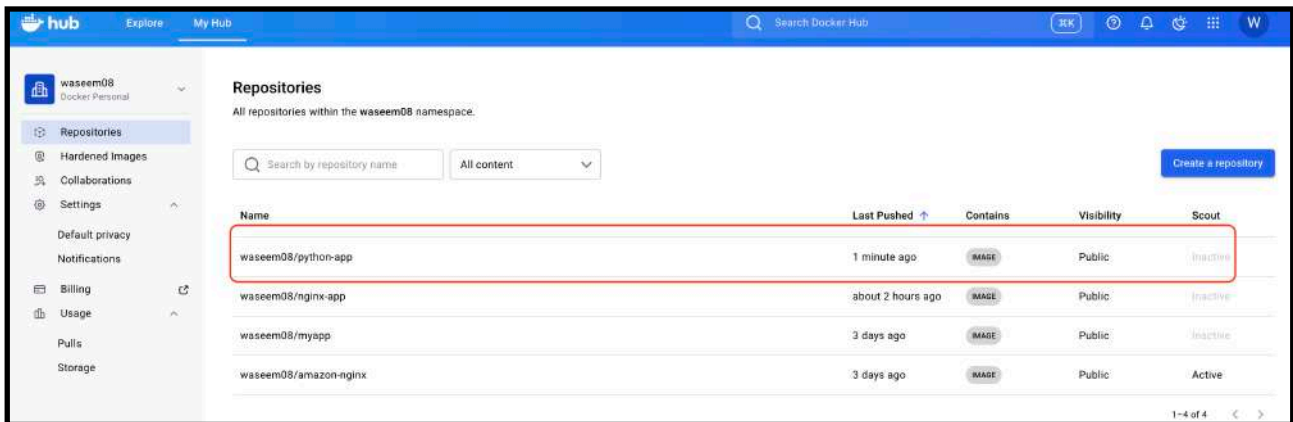
Verify :

Open the docker hub

—> Docker image builds

- Image pushed to Docker Hub:

waseem08/python-app:latest



Step;4

pull the image from docker hub to local and run

```
docker pull waseem08/python-app:latest
```

—> docker images (to verify)

```
[root@ip-172-31-20-146 ~]# docker pull waseem08/python-app:latest
latest: Pulling from waseem08/python-app
0e29546d541c: Pull complete
9b829c73b52b: Pull complete
cb5b7ae36172: Pull complete
6494e4811622: Pull complete
6f9f74896dfa: Pull complete
5e3b1213efc5: Pull complete
9fddfdc56334: Pull complete
404f02044bac: Pull complete
c4f42be2be53: Pull complete
7ad8f8aa8f9d: Pull complete
4339c32b75d3: Pull complete
4f4fb700ef54: Pull complete
Digest: sha256:b4aaf61661f9f0facf315b905f5536c107c7dec2e22f2178b9e338eefab18934
Status: Downloaded newer image for waseem08/python-app:latest
docker.io/waseem08/python-app:latest
[root@ip-172-31-20-146 ~]# docker images'
> ^C
[root@ip-172-31-20-146 ~]# docker images

```

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
waseem08/python-app	latest	9e3e150a2a31	11 minutes ago	913MB

STEP 5 :Run the Docker image

Your app exposes **port 8080**, so run:

```
docker run -d -p 8080:8080 --name python-app waseem08/python-app:latest
```

Check container:

```
docker ps
```

```
[root@ip-172-31-20-146 ~]# docker run -d -p 8085:8080 --name python-app waseem08/python-app:latest
9393fbb3057e0f834ed9c13e11d392e4f21f92a0f83aa8edf521233ae304486e
[root@ip-172-31-20-146 ~]# docker ps

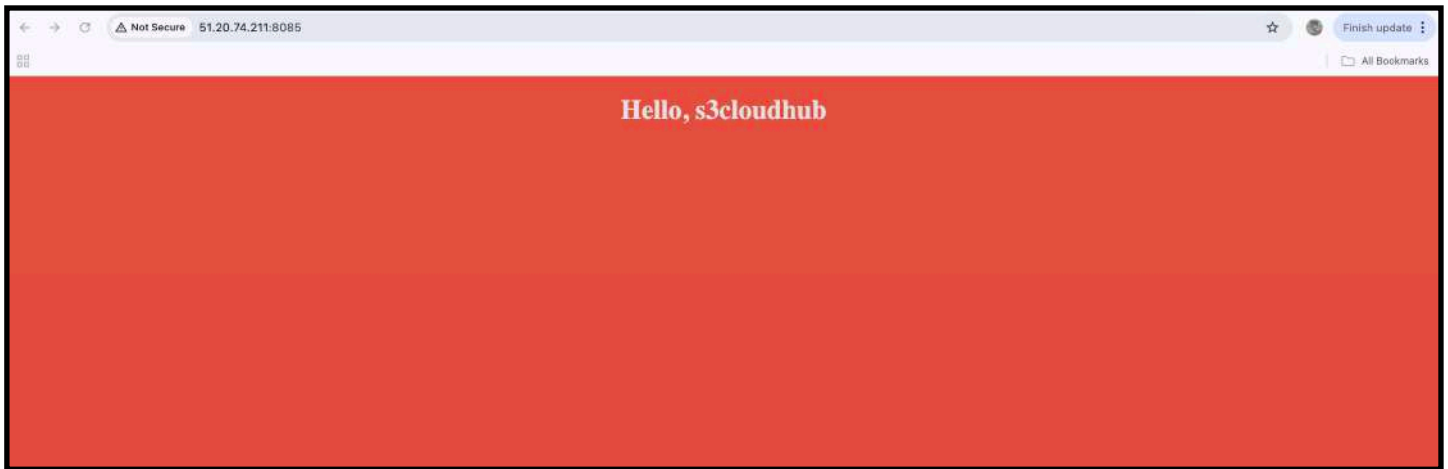
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS
9393fbb3057e	waseem08/python-app:latest	"python app.py"	3 minutes ago	Up 3 minutes	0.0.0.0:8085->8080/tcp, :::8085

```
[root@ip-172-31-20-146 ~]#
```

STEP 6: Access the application
unning on EC2:

http://<EC2_PUBLIC_IP>:8080



Source Codes: <https://github.com/betawins/docker-tasks.git>

A) From the frontend source code, write a Dockerfile, build a Docker image, run it, and push that image to your Docker registry.

Step1:

Clone your fork (or original if allowed)

```
git clone https://github.com/betawins/docker-tasks.git
```

```
[root@ip-172-31-20-146 ~]# git clone https://github.com/betawins/docker-tasks.git
Cloning into 'docker-tasks'...
remote: Enumerating objects: 146, done.
remote: Counting objects: 100% (146/146), done.
remote: Compressing objects: 100% (123/123), done.
remote: Total 146 (delta 12), reused 141 (delta 9), pack-reused 0 (from 0)
Receiving objects: 100% (146/146), 22.11 MiB | 39.37 MiB/s, done.
Resolving deltas: 100% (12/12), done.
[root@ip-172-31-20-146 ~]# cd docker-tasks/
[root@ip-172-31-20-146 docker-tasks]# ls
Frontend_based_source  Java_based_source  NodeJs_based_source  README.md
[root@ip-172-31-20-146 docker-tasks]#
```

2 Go to frontend source folder

```
cd Frontend_based_source
```

```
# Use lightweight Nginx image
FROM nginx:alpine

# Set working directory
WORKDIR /usr/share/nginx/html

# Remove default Nginx content
RUN rm -rf ./*

# Copy frontend project files
COPY . .

# Expose port 80
EXPOSE 80

# Start Nginx
CMD ["nginx", "-g", "daemon off;"]
```


3 Create Dockerfile

vi Dockerfile

→ save.

Then:

docker build -t waseem08/frontend-app .

```
[root@ip-172-31-20-146 Frontend_based_source]# vi Dockerfile
[root@ip-172-31-20-146 Frontend_based_source]# docker build -t waseem08/frontend-app .
[+] Building 1.4s (10/10) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 368B
=> [internal] load metadata for docker.io/library/nginx:alpine
=> [auth] library/nginx:pull token for registry-1.docker.io
=> [internal] load .dockerignore
=> => transferring context: 2B
=> CACHED [1/4] FROM docker.io/library/nginx:alpine@sha256:1d13701a5f9f3fb01aaa88cef2344d65b6b5bf6b7d9f
=> [internal] load build context
=> => transferring context: 39.90kB
=> [2/4] WORKDIR /usr/share/nginx/html
=> [3/4] RUN rm -rf ./*
=> [4/4] COPY . .
=> exporting to image
=> => exporting layers
=> => writing image sha256:248eb68968315524f236d9b27f2afc478283b3b6363fa8fdecce840e8dfe2044
=> => naming to docker.io/waseem08/frontend-app
[root@ip-172-31-20-146 Frontend_based_source]#
```

Run the container :

docker run -d -p 8081:80 waseem08/frontend-app

```
[root@ip-172-31-20-146 Frontend_based_source]# docker run -d -p 8081:80 waseem08/frontend-app
ec077d59a884150cae867002ce27edd379fd001622d907de354a438daace4591
[root@ip-172-31-20-146 Frontend_based_source]# docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS
ec077d59a884	waseem08/frontend-app	"/docker-entrypoint..."	4 seconds ago	Up 3 seconds	0.0.0.0:8081->80/tcp

PUSH IMAGE TO DOCKER HUB

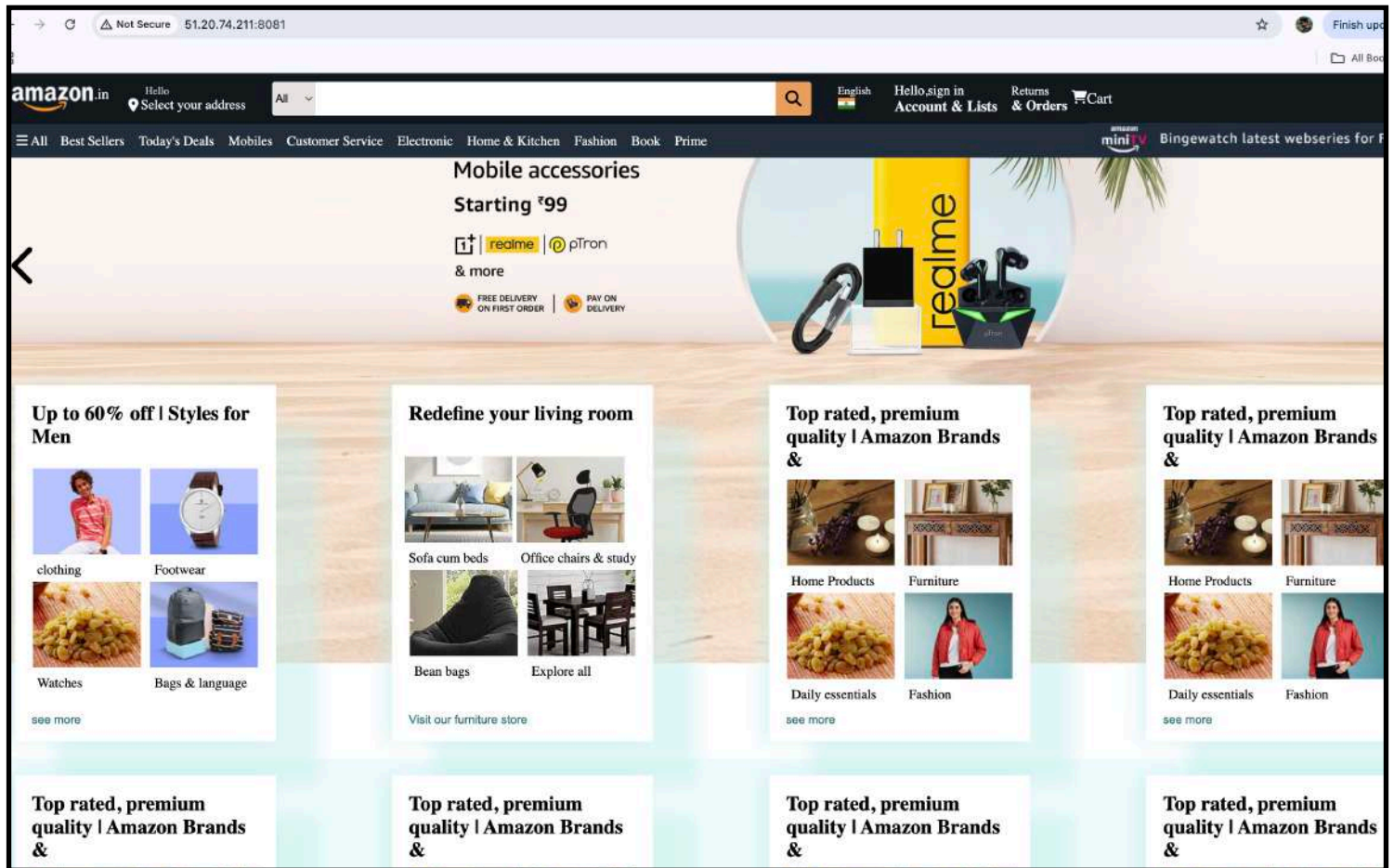
`docker push waseem08/frontend-app`

```
[root@ip-172-31-20-146 Frontend_based_source]# docker push waseem08/frontend-app
Using default tag: latest
The push refers to repository [docker.io/waseem08/frontend-app]
cc309a4903d9: Pushed
72aa1ae0255a: Pushed
5f70bf18a086: Mounted from waseem08/python-app
da3ac26fdf0f: Mounted from waseem08/nginx-app
7b2903554e63: Mounted from waseem08/nginx-app
6c0e59fd138a: Mounted from waseem08/nginx-app
aa0fc249df10: Mounted from waseem08/nginx-app
f35bcec50d8c: Mounted from waseem08/nginx-app
660f9a93104f: Mounted from waseem08/nginx-app
53998a5033c3: Mounted from waseem08/nginx-app
989e799e6349: Mounted from waseem08/nginx-app
latest: digest: sha256:27127f47cafa872e7a64e398ad6ce9dcd295132493c51762ce2580a6ab844ae1 size: 2610
[root@ip-172-31-20-146 Frontend_based_source]#
```

Now verify

Open in browser:

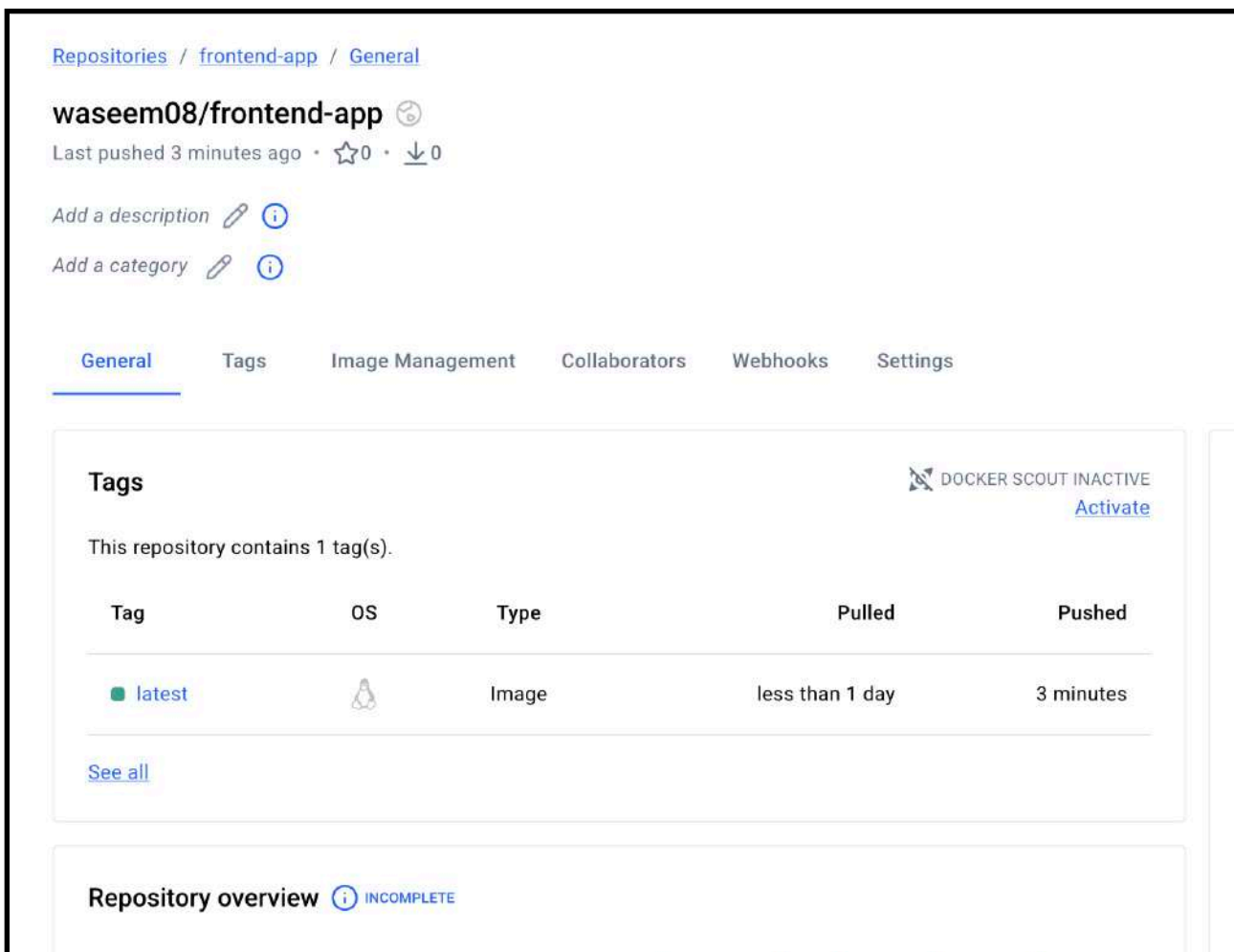
`http://publicip:8081`



—>now check your docker hub



—> repository



Steps used :

Clone the repository

```
git clone https://github.com/betawins/docker-tasks.git
```

Go to frontend source directory

```
cd docker-tasks/Frontend_based_source
```

Build Docker image

```
docker build -t waseem08/frontend-app .
```

Run Docker container

```
docker run -d -p 8081:80 --name frontend waseem08/  
frontend-app
```

Push image to Docker Hub

docker push waseem08/frontend-app

B) From the Java-based source code, write a Dockerfile, build, run, and push the image to the Docker registry.

cd docker-tasks/Java_based_source

Check files:

ls

(You should see pom.xml, src/,

```
[root@ip-172-31-20-146 ~]# cd docker-tasks/
[root@ip-172-31-20-146 docker-tasks]# ls
Frontend_based_source  Java_based_source  NodeJs_based_source  README.md
[root@ip-172-31-20-146 docker-tasks]# cd Java_based_source/
[root@ip-172-31-20-146 Java_based_source]# ls
pom.xml  src
[root@ip-172-31-20-146 Java_based_source]#
```

—>Create Dockerfile (Java – Maven build)

vi Dockerfile

```
# ----- Build stage -----
FROM maven:3.9.9-eclipse-temurin-17 AS build
WORKDIR /app
COPY pom.xml .
COPY src ./src
RUN mvn clean package -DskipTests

# ----- Runtime stage -----
FROM eclipse-temurin:17-jre
WORKDIR /app
COPY --from=build /app/target/*.jar app.jar
EXPOSE 8080
CMD ["java", "-jar", "app.jar"]
```


:save

—> Build Docker image

docker build -t waseem08/java-app .
Verify:

docker images

```
[root@ip-172-31-20-146 Java_based_source]# vi Dockerfile
[root@ip-172-31-20-146 Java_based_source]# docker build -t waseem08/java-app .
[+] Building 29.1s (16/16) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 426B
=> [internal] load metadata for docker.io/library/eclipse-temurin:17-jre
=> [internal] load metadata for docker.io/library/maven:3.9.9-eclipse-temurin-17
=> [auth] library/eclipse-temurin:pull token for registry-1.docker.io
=> [auth] library/maven:pull token for registry-1.docker.io
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [build 1/5] FROM docker.io/library/maven:3.9.9-eclipse-temurin-17@sha256:f58d59b6273e785ac0a4477f6e9b5b
=> => resolve docker.io/library/maven:3.9.9-eclipse-temurin-17@sha256:f58d59b6273e785ac0a4477f6e9b5b1d773
=> => sha256:f58d59b6273e785ac0a4477f6e9b5b1d7731c75b906c0f7b34076f1851318cc 7.86kB / 7.86kB
=> => sha256:82e47241881f23ad774f5db8829efca15758e8fdb5b1d64ea9f8d6420a85068e 2.90kB / 2.90kB
=> => sha256:d9d352c11bbd3880007953ed6eec1cbace76898828f3434984a0ca60672fdf5a 29.72MB / 29.72MB
=> => sha256:7bbafedfaf521e89144400b08c3dbacde18a658508d1b700f2b98bf123959ebe 22.95MB / 22.95MB
=> => sha256:d0841781763b62e537284c5b6e5efe3f3b69b03a65b64d46a5dec054d9a35279 9.73kB / 9.73kB
=> => sha256:b61b7669ae30b45ca88696ba9b505614969b7b22fcfb0131a0fe11a21cf6028 144.65MB / 144.65MB
=> => extracting sha256:d9d352c11bbd3880007953ed6eec1cbace76898828f3434984a0ca60672fdf5a
=> => sha256:d9815e77b54f33dbcd213d596eeaad209e5255afa9c22666ba991c8c1daf1211 2.28kB / 2.28kB
```

Run the container

docker run -d -p 8082:8080 --name java-app waseem08/java-app
Check:

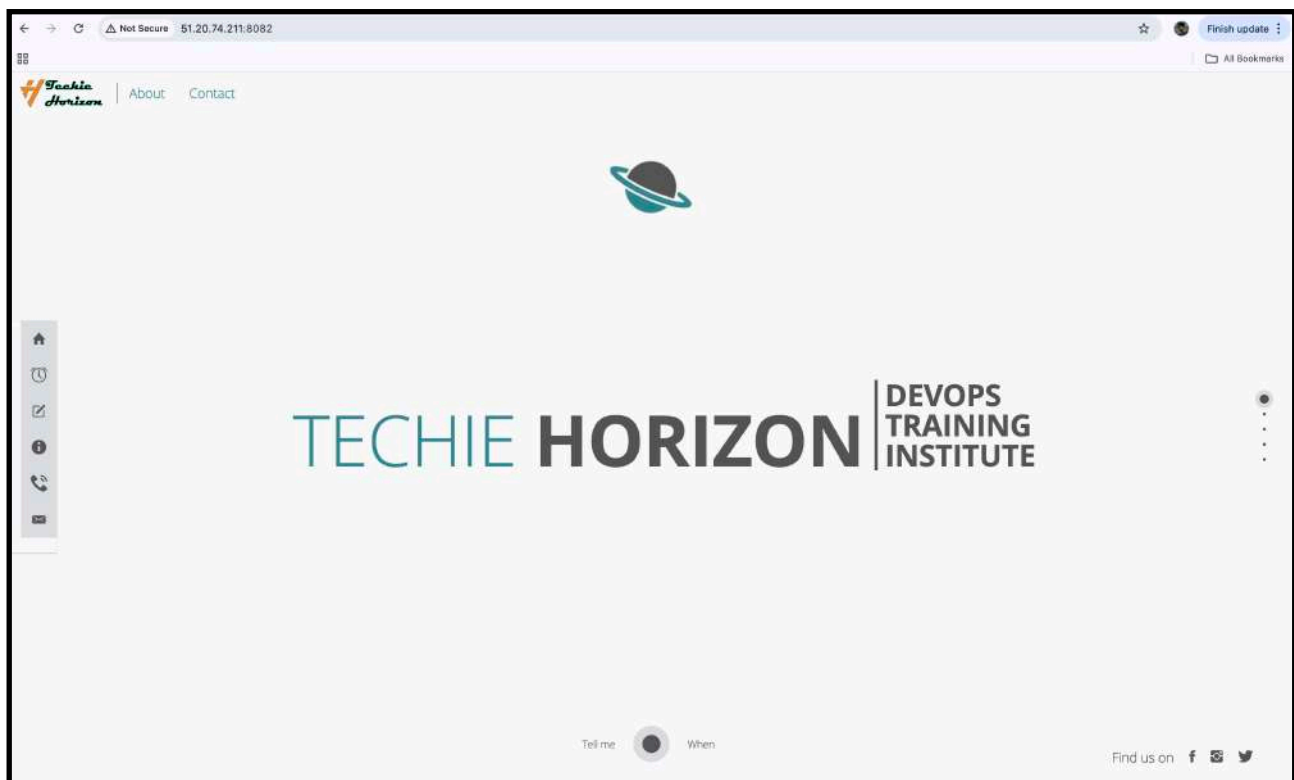
docker ps

```
[root@ip-172-31-20-146 Java_based_source]# docker run -d -p 8082:8080 --name java-app waseem08/java-app
aaadd1d5f9cada6551f2b840f1ddf78c76402457b60d057a460214a5694ef955
[root@ip-172-31-20-146 Java_based_source]# docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS
aaadd1d5f9ca	waseem08/java-app	"/_cacert_entrypoin..."	5 seconds ago	Up 5 seconds	0.0.0.0:8082->8080/tcp, :::8082

Open (if it's a web app):

`http://<EC2_PUBLIC_IP>:8082`



Push image to Docker Hub

```
docker login
```

```
docker push waseem08/java-app
```



```

jenkins/jenkins [LS] 55bc455e1f05 2 weeks ago
[root@ip-172-31-20-146 Java_based_source]# docker push waseem08/java-app
Using default tag: latest
The push refers to repository [docker.io/waseem08/java-app]
94a0933de43b: Pushed
6177b09381ec: Pushed
80bffc412b8: Mounted from library/eclipse-temurin
467f47db56d3: Mounted from library/eclipse-temurin
92dfe22bf04e: Mounted from library/eclipse-temurin
75e511ae6fd4: Mounted from library/eclipse-temurin
123a078714d5: Mounted from waseem08/amazon-nginx
latest: digest: sha256:b90b3aba358682e993208f09e1dd9485cab533340d47b97182353ffe45becbda size: 1786
[root@ip-172-31-20-146 Java_based_source]#

```

Verify in docker hub

—> repositories

[Repositories](#) / [java-app](#) / [General](#)

waseem08/java-app

Last pushed 1 minute ago ·  0 ·  0

Add a description  

Add a category  

[General](#)
[Tags](#)
[Image Management](#)
[Collaborators](#)
[Webhooks](#)
[Settings](#)

Tags

 DOCKER SCOUT INACTIVE [Activate](#)

This repository contains 1 tag(s).

Tag	OS	Type	Pulled	Pushed
 latest		Image	less than 1 day	1 minute

[See all](#)

Repository overview INCOMPLETE

Java application was containerized using Docker and pushed to Docker Hub.

c) From the **Node.js-based source code**, write a Dockerfile, build with tag **v1**, run, and push it to the Docker registry.

Repo: <https://github.com/betawins/docker-tasks.git>

Folder: NodeJs_based_source

Goal: Dockerfile → build **with tag v1** → run → push

—> cd docker-tasks/NodeJs_based_source
ls

You should see files like:

package.json

index.js / app.js

```
[root@ip-172-31-20-146 docker-tasks]# cd NodeJs_based_source/  
[root@ip-172-31-20-146 NodeJs_based_source]# ls  
package-lock.json package.json public src  
[root@ip-172-31-20-146 NodeJs_based_source]#
```

—> Create Dockerfile (inside NodeJs_based_source)

vi Dockerfile

```
# Use Node.js image
FROM node:18-alpine

# Set working directory
WORKDIR /app

# Copy dependency files
COPY package*.json ./

# Install dependencies
RUN npm install

# Copy application source
COPY . .

# Expose React default port
EXPOSE 3000

# Start the application
CMD ["npm", "start"]

~
~
```

—> save

Build Docker image (IMPORTANT: tag v1)

```
docker build -t waseem08/node-app:v1 .
Check:
```

```
docker images
```

```
[root@ip-172-31-20-146 NodeJs_based_source]# docker build -t waseem08/node-app:v1 .
[+] Building 35.9s (11/11) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 388B
=> [internal] load metadata for docker.io/library/node:18-alpine
=> [auth] library/node:pull token for registry-1.docker.io
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [1/5] FROM docker.io/library/node:18-alpine@sha256:8d6421d663b4c28fd3ebc498332f249011d118945588d0a35cb9bc4b8ca
=> => resolve docker.io/library/node:18-alpine@sha256:8d6421d663b4c28fd3ebc498332f249011d118945588d0a35cb9bc4b8ca
=> => sha256:8d6421d663b4c28fd3ebc498332f249011d118945588d0a35cb9bc4b8ca09d9e 7.67kB / 7.67kB
=> => sha256:929b04d7c782f04f615cf785488fed452b6569f87c73ff666ad553a7554f0006 1.72kB / 1.72kB
=> => sha256:ee77c6cd7c1886ecc802ad6cedef3a8ec1ea27d1fb96162bf03dd3710839b8da 6.18kB / 6.18kB
=> => sha256:f18232174bc91741fdf3da96d85011092101a032a93a388b79e99e69c2d5c870 3.64MB / 3.64MB
=> => sha256:dd71dde834b5c203d162902e6b8994cb2309ae049a0eabc4efea161b2b5a3d0e 40.01MB / 40.01MB
=> => sha256:1e5a4c89cee5c0826c540ab06d4b6b491c96eda01837f430bd47f0d26702d6e3 1.26MB / 1.26MB
=> => extracting sha256:f18232174bc91741fdf3da96d85011092101a032a93a388b79e99e69c2d5c870
=> => sha256:25ff2da83641908f65c3a74d80409d6b1b62ccfaab220b9ea70b80df5a2e0549 446B / 446B
=> => extracting sha256:dd71dde834b5c203d162902e6b8994cb2309ae049a0eabc4efea161b2b5a3d0e
=> => extracting sha256:1e5a4c89cee5c0826c540ab06d4b6b491c96eda01837f430bd47f0d26702d6e3
=> => extracting sha256:25ff2da83641908f65c3a74d80409d6b1b62ccfaab220b9ea70b80df5a2e0549
=> [internal] load build context
=> => transferring context: 4.13MB
=> [2/5] WORKDIR /app
```

```
[root@ip-172-31-20-146 NodeJs_based_source]# docker images
REPOSITORY          TAG          IMAGE ID          CREATED          SIZE
waseem08/node-app   v1           59413f2fd75a     57 seconds ago   510MB
```

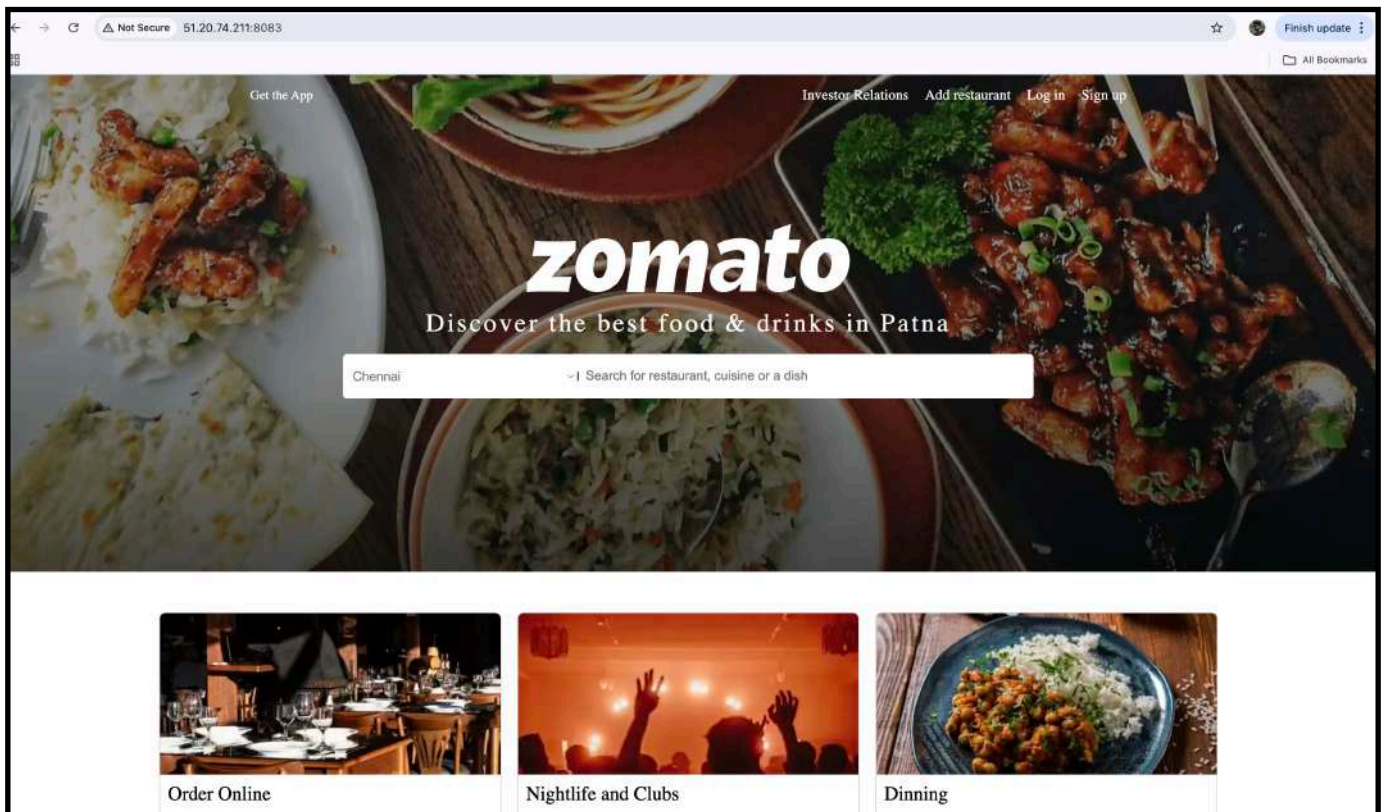
Run the container

```
docker run -d -p 8083:3000 --name node-app waseem08/node-app:v1
```

```
jenkins/ jenkins          ecs          55b753e1f05      2 weeks ago      451MB
[root@ip-172-31-20-146 NodeJs_based_source]# docker run -d -p 8083:3000 --name node-app waseem08/node-app:v1
d5b7edb71e0fc8467634c8b75cb605e1704a128037720c71297d49113d4721ae
[root@ip-172-31-20-146 NodeJs_based_source]# docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED          STATUS          PORTS
MES
d5b7edb71e0f   waseem08/node-app:v1   "docker-entrypoint.s..." 4 seconds ago   Up 3 seconds   0.0.0.0:8083->3000/tcp
```

Open in browser:

http://<EC2_PUBLIC_IP>:8083



Docker tag the image and push

Push image to Docker Hub

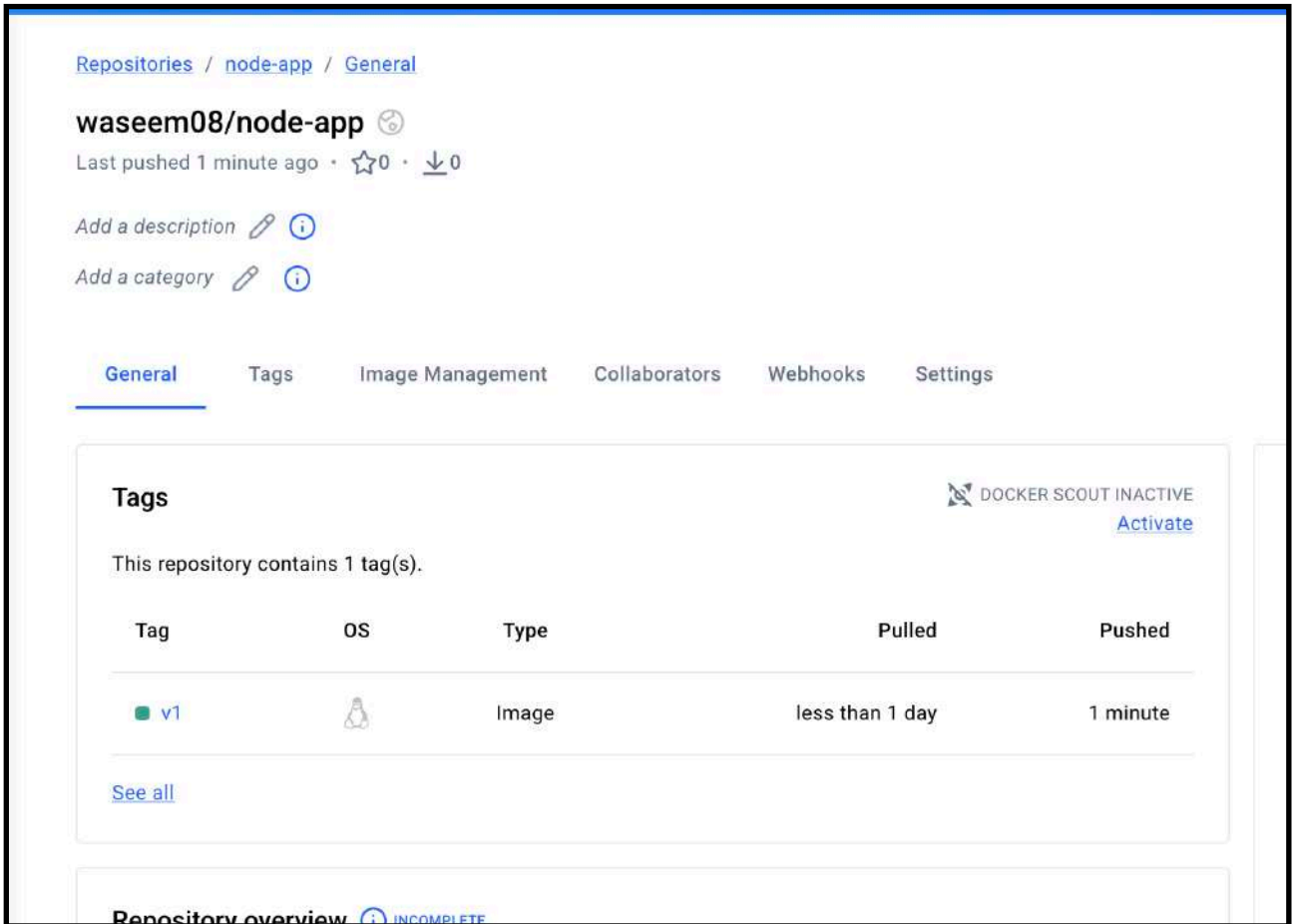
```
docker login
docker push waseem08/node-app:v1
```

```
[root@ip-172-31-20-146 NodeJs_based_source]# docker push waseem08/node-app:v1

The push refers to repository [docker.io/waseem08/node-app]
9a06d2524a93: Pushed
16a832fccc6d: Pushed
2213c39d9e9b: Pushed
49db53a00466: Pushed
82140d9a70a7: Mounted from library/node
f3b40b0cdb1c: Mounted from library/node
0b1f26057bd0: Mounted from library/node
08000c18d16d: Mounted from library/node
v1: digest: sha256:8e74d3be7a6f412cf74ba21bb28945a9da30fa2f9502c628789a259638d0761d size: 1998
[root@ip-172-31-20-146 NodeJs_based_source]#
```

Verify by docker hub

Repositories



The screenshot shows the Docker Hub interface for a repository named 'waseem08/node-app'. The page is titled 'Repositories / node-app / General'. Below the title, it shows 'Last pushed 1 minute ago' and '0 stars, 0 downloads'. There are links to 'Add a description' and 'Add a category'. The 'General' tab is selected, showing a 'Tags' section with a table of repository tags. The table has columns for 'Tag', 'OS', 'Type', 'Pulled', and 'Pushed'. One tag is listed: 'v1' with OS 'linux', Type 'Image', Pulled 'less than 1 day', and Pushed '1 minute'. There is a 'See all' link below the table. At the bottom, there is a 'Repository overview' section with an 'INCOMPLETE' status.

Repositories / node-app / General

waseem08/node-app 

Last pushed 1 minute ago · ☆0 · ↓0

[Add a description](#)  

[Add a category](#)  

General Tags Image Management Collaborators Webhooks Settings

Tags  DOCKER SCOUT INACTIVE [Activate](#)

This repository contains 1 tag(s).

Tag	OS	Type	Pulled	Pushed
 v1		Image	less than 1 day	1 minute

[See all](#)

Repository overview  INCOMPLETE

Last task challenge :

Write a docker-compose file to set up WordPress with a MySQL database.

Step 1: Install Docker Compose v2

Run these commands step by step:

1 Create the plugin directory

```
sudo mkdir -p /usr/local/lib/docker/cli-plugins
```

2 Download the latest Docker Compose binary

```
sudo curl -SL https://github.com/docker/compose/releases/latest/download/docker-compose-  
linux-x86_64 \\\n-o /usr/local/lib/docker/cli-plugins/docker-compose
```

3 Give execute permission

```
sudo chmod +x /usr/local/lib/docker/cli-plugins/docker-compose
```

```
[root@ip-172-31-20-146 ~]# cd /usr/local/lib/docker/cli-plugins
[root@ip-172-31-20-146 ~]# sudo mkdir -p /usr/local/lib/docker/cli-plugins
[root@ip-172-31-20-146 ~]# sudo curl -SL https://github.com/docker/compose/releases/latest/download/docker-compose-linux-x86_64 \
-o /usr/local/lib/docker/cli-plugins/docker-compose
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left   Speed
  0     0    0     0    0     0      0     0  --:--:-- --:--:-- --:--:--    0
  0     0    0     0    0     0      0     0  --:--:-- --:--:-- --:--:--    0
100 29.8M 100 29.8M    0     0  65.2M      0  --:--:-- --:--:-- --:--:--  65.2M
[root@ip-172-31-20-146 ~]#
```

```
sudo chmod +x /usr/local/lib/docker/cli-plugins/docker-  
compose
```

```
100 29.8M 100 29.8M    0     0  65.2M      0  --:--:-- --:--:-- --:--:--  65.2M
[root@ip-172-31-20-146 ~]# sudo chmod +x /usr/local/lib/docker/cli-plugins/docker-compose
[root@ip-172-31-20-146 ~]#
```


Verify:

```
docker compose version
```

```
[root@ip-172-31-20-146 ~]# docker compose version
Docker Compose version v5.0.2
[root@ip-172-31-20-146 ~]#
```

STEP 2: Create docker-compose file

Create a new folder (anywhere you like):

```
mkdir wordpress-compose
cd wordpress-compose
vi docker-compose.yml
```

```
version: '3.8'

services:
  db:
    image: mysql:5.7
    container_name: wordpress-db
    restart: always
    environment:
      MYSQL_DATABASE: wordpress
      MYSQL_USER: wpuser
      MYSQL_PASSWORD: wppassword
      MYSQL_ROOT_PASSWORD: rootpassword
    volumes:
      - db_data:/var/lib/mysql

  wordpress:
    image: wordpress:latest
    container_name: wordpress-app
    restart: always
    ports:
      - "8084:80"
    environment:
      WORDPRESS_DB_HOST: db:3306
      WORDPRESS_DB_USER: wpuser
      WORDPRESS_DB_PASSWORD: wppassword
      WORDPRESS_DB_NAME: wordpress
    depends_on:
      - db
    volumes:
      - wp_data:/var/www/html

volumes:
  db_data:
  wp_data:
```

—> save and exit

Step:3

Start containers

`docker compose up -d`

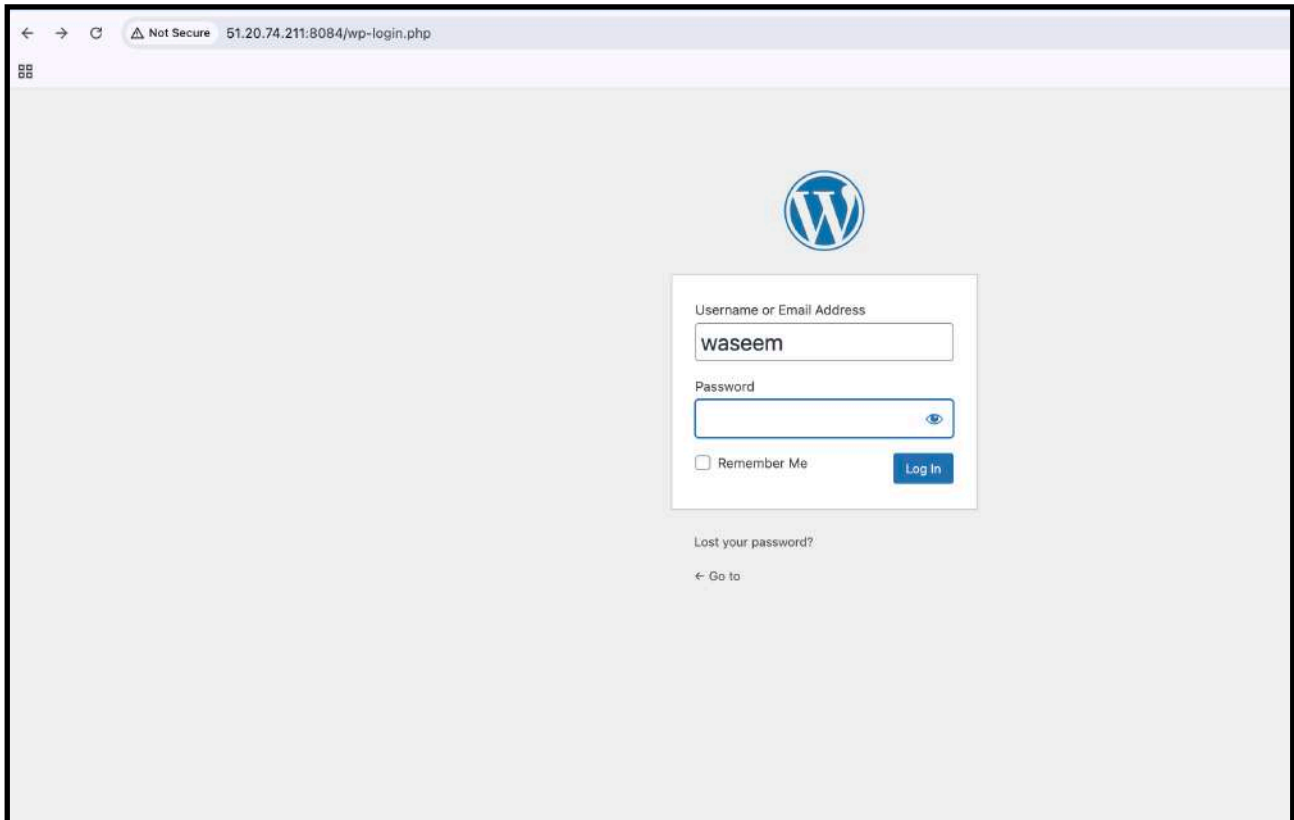
```
[root@ip-172-31-20-146 wordpress-compose]# docker compose up -d
WARN[0000] /root/wordpress-compose/docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion
[+] up 42/42
  ✓ Image mysql:5.7          Pulled          14.4s
  ✓ Image wordpress:latest   Pulled          10.7s
  ✓ Network wordpress-compose_default Created        0.1s
  ✓ Volume wordpress-compose_wp_data Created         0.0s
  ✓ Volume wordpress-compose_db_data Created         0.0s
  ✓ Container wordpress-db    Created         3.3s
  ✓ Container wordpress-app    Created         0.0s
[root@ip-172-31-20-146 wordpress-compose]# docker ps
CONTAINER ID   IMAGE                COMMAND                  CREATED        STATUS        PORTS                               NAMES
8701788c1987   wordpress:latest     "docker-entrypoint.s..." 19 seconds ago Up 17 seconds 0.0.0.0:8084->80/tcp, :::8084->80/tcp wordpress-app
```

Verify

`docker ps`

Access WordPress

`http://<EC2-IP>:8084`



—> set username password login

